

SoPC Abgabedokument

Florian Breitwieser & Jakob Zimmermann

2. Juni 2026

Inhaltsverzeichnis

1	Addressierungsbeispiel	3
1.1	Einleitung	3
1.2	Aufgabenstellung	3
1.3	Addr_Slave Code	4
1.4	Addr_Slave Testbench	5
1.5	Master Architektur und Code	7
1.6	Testbench	11
1.7	Metastabilität	13
1.7.1	Ursachen	13
1.7.2	Auswirkungen	13
1.7.3	Vermeidung	13
2	UART TX	14
3	Aufgabe 4: UART RX	22
4	Aufgabe 5.2: Adressierungsarten und Alignment	31
4.1	Assemblercode der drei Zuweisungen	32
4.2	Analyse: Unterschiede und Erklärung	34
5	Seven-Segment-Anzeige (Aufgabe 6)	35
5.1	Grundsätzlicher Aufbau	35
5.2	VHDL-Modul: iceduino_sevensegment.vhd	35
5.3	Integration in das Top-Level	38
5.3.1	Neues Slave-Response-Signal	38
5.3.2	Instanziierung des Seven-Segment-Moduls	39
5.3.3	Erweiterung des Bus-Multiplexers	39
5.4	Software-Integration	40
5.4.1	HAL: iceduino_sevensegment.h und iceduino_sevensegment.c	40

5.4.2	Memory-Mapped-I/O-Adresse	41
5.4.3	Anwendungsprogramm: <code>main.c</code>	41
5.5	Build-System-Anpassungen	41
5.5.1	<code>osflow/boards/iceduino/Makefile</code>	42
5.5.2	<code>simulation/Makefile</code>	42
5.6	Simulation	42
5.7	Pin-Zuweisung und Hardware-Aufbau	44
6	WS2812 Ansteuerung Entwicklung des Aserial Moduls (Aufgabe 7)	46
6.1	Implementierung des Aserial Moduls	46
6.2	Testbench für das Aserial Modul	47
6.3	Simulationsergebnisse	49

Abbildungsverzeichnis

1	Simulation des Address-Slave mit Testbench	6
2	Blockschaltbild des Address-Master und Slave-Anbindung	7
3	Zustandsdiagramm des Address-Master	10
4	Simulation des Address-Master mit Slave	12
5	UART TX Blockdiagramm	14
6	UART TX Statemaschine	15
7	UART TX Simulation 1	21
8	UART TX Simulation 2	21
9	UART TX Simulation 3	22
10	UART RX Blockdiagramm	23
11	GTKWave-Simulation des UART RX Moduls: Verlauf der Signale <code>state</code> , <code>uart_in</code> und <code>led</code>	30
12	GTKWave-Simulation: Zeitlicher Verlauf von <code>reg_sevensegment</code> mit Zähl- sequenz 0–9. Die Änderungen beginnen nach ca. 170 μ s, der Zyklus beträgt ca. 720 ns.	43
13	Foto des Aufbaus der realen Hardware: FPGA-Board mit angeschlossener 7-Segment-Anzeige. Die Anzeige ist über Vorwiderstände mit den FPGA- Ausgängen verbunden, die Common-Kathode liegt an GND.	44
14	Simulation des Aserial Moduls. Die Bitfolge 1010 wird in MSB-First Reihenfolge gesendet.	49
15	Timing des ersten Bits	50
16	Timing des zweiten Bits	50
17	Timing des dritten Bits	50
18	Timing des vierten Bits	51

1 Adressierungsbeispiel

1.1 Einleitung

Dieses Dokument beschreibt die Umsetzung des Adressierungsbeispiels als Teil der Aufgabe 3. Es fasst die Architektur des Address-Slave und des Address-Master zusammen, dokumentiert die Simulation und beschreibt Metastabilitätseffekte.

1.2 Aufgabenstellung

Ziel ist die Implementierung eines adressierbaren LED-Slave-Moduls für das iceduino-Board und eines synthetisierbaren Masters, der die LEDs in einem Lauflicht ansteuert. Die Abgabeanforderungen umfassen:

- Slave Code und erläuternder Text zur Funktion
- Slave Testbench Code und erläuternder Text zur Funktion, inklusive asserts und Hinweis auf nicht-synthetisierbaren Testbench-Code
- Simulation Slave mit Testbench (Screenshot + Analyse)
- Master Blockschaltbild inkl. Slave-Anbindung
- Master Code
- Master Simulation mit Slave
- Erläuterung der Simulationsergebnisse und Master-Statemachine
- Metastabilitätsbetrachtung: Ursache, Auswirkung und Vermeidung

1.3 Addr_Slave Code

Der folgende Code zeigt das Low-Level-Verhalten des Slave-Moduls. Es verarbeitet den Reset, liest die Adresse und schreibt bei gesetztem `we` das Datensignal `d` in das entsprechende LED-Register.

```
1  library ieee;
2  use IEEE.std_logic_1164.all;
3  use IEEE.numeric_std.all;
4
5  entity addr_slave is
6      port(
7          clk: in std_logic;
8          we : in std_logic;
9          d  : in std_logic;
10         rst : in std_logic;
11         addr : in std_logic_vector(2 downto 0);
12         leds : out std_logic_vector(7 downto 0)
13     );
14 end entity;
15
16 architecture addr_slave_a of addr_slave is
17 begin
18     process(clk, rst) is
19     begin
20         if rst = '1' then
21             leds <= (others => '0');
22         elsif rising_edge(clk) then
23             if we = '1' then
24                 leds(to_integer(unsigned(addr))) <= d;
25             end if;
26         end if;
27     end process;
28 end architecture addr_slave_a;
```

Bei steigender Flanke des Taktsignals `clk` und aktivem Reset (`rst = '1'`) werden alle LEDs gelöscht. Wenn `we` gesetzt ist, wird das Datenbit `d` an der Position geschrieben, die durch die Adresse `addr` bestimmt wird. Die LEDs werden als 8-Bit-Vektor ausgegeben, wobei jedes Bit eine LED repräsentiert.

1.4 Addr_Slave Testbench

Die Testbench prüft die korrekte Adressierung des Slave-Moduls für mehrere Adressen und enthält Assert-Anweisungen für die LED-Ausgabe.

```
1  library ieee;
2  use IEEE.std_logic_1164.all;
3  use IEEE.numeric_std.all;
4
5  entity addr_slave_tb is
6  end entity;
7
8  architecture addr_slave_tb_a of addr_slave_tb is
9      signal clk : std_logic := '0';
10     signal we : std_logic := '0';
11     signal d : std_logic := '0';
12     signal reset : std_logic := '0';
13     signal addr : std_logic_vector(2 downto 0) := (others => '0');
14     signal leds : std_logic_vector(7 downto 0) := (others => '0');
15 begin
16     clk <= not clk after 10 ns; -- 50 MHz
17
18     addr_slave_instance : entity work.addr_slave
19         port map(
20             clk => clk,
21             we => we,
22             d => d,
23             reset => reset,
24             addr => addr,
25             leds => leds
26         );
27
28     process is
29     begin
30         reset <= '1';
31         wait for 30 ns;
32         reset <= '0';
33
34         addr <= (others => '0');
35         wait for 10 ns;
36         d <= '1';
37         we <= '1';
38
39         wait for 80 ns;
40         assert (leds = "00000001") report "R1: LEDS falsch" severity error;
41         assert (addr = "000") report "R1: b0 falsch" severity error;
42         report "R1: OK" severity note;
43         we <= '0';
44         wait for 1 ns;
45         d <= '0';
```

```

46
47     wait for 200 ns;
48     addr <= (others => '1');
49     wait for 10 ns;
50     d <= '1';
51     we <= '1';
52
53     wait for 80 ns;
54     assert (leds = "10000001") report "R2: LEDS falsch" severity error;
55     assert (addr = "111") report "R2: addr falsch" severity error;
56     report "R2: OK" severity note;
57     we <= '0';
58     wait for 1 ns;
59     d <= '0';
60
61     wait for 200 ns;
62     addr <= "010";
63     wait for 10 ns;
64     d <= '1';
65     we <= '1';
66
67     wait for 80 ns;
68     assert (leds = "10000101") report "R3: LEDS falsch" severity error;
69     report "R3: OK" severity note;
70     we <= '0';
71     wait for 200 ns;
72     d <= '0';
73 end process;
74 end architecture addr_slave_tb_a;

```

Alle Asserts konnten erfolgreich durchlaufen werden. Der addr-slave funktioniert einwandfrei.

Alle 'wait' und 'after' Anweisungen sind nicht-synthetisierbar und dienen ausschließlich der Simulation.

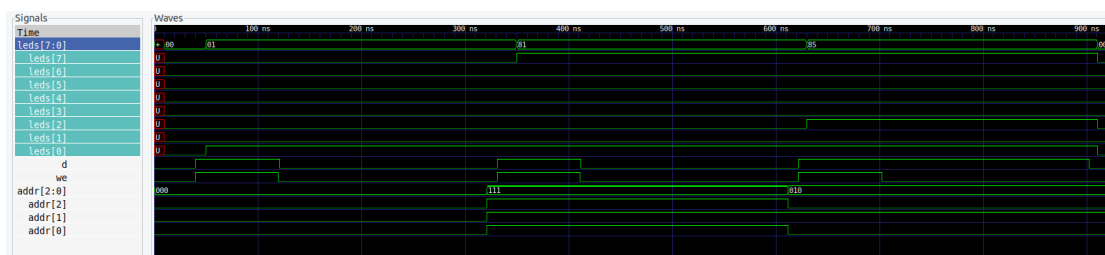


Abbildung 1: Simulation des Address-Slave mit Testbench

Die Simulation zeigt die korrekte Adressierung und LED-Ausgabe für die getesteten Adressen. Bei Adresse '000' leuchtet die erste LED, bei '111' die letzte LED, und bei '010' die dritte LED. Alle Asserts bestätigen die erwarteten Ergebnisse.

1.5 Master Architektur und Code

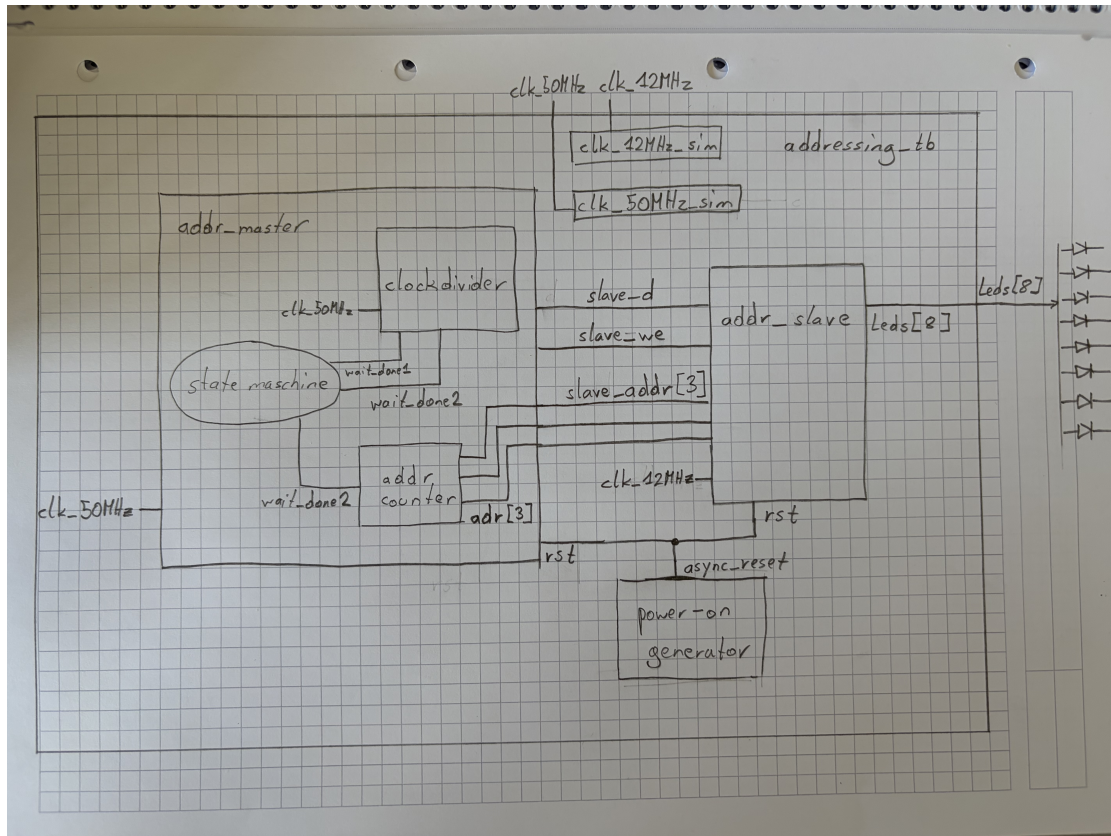


Abbildung 2: Blockschaltbild des Address-Master und Slave-Anbindung

Das Bild zeigt die Zusammenschaltung des Address-Master mit dem Address-Slave. Der Master generiert die Signale `adr`, `we` und `d`, die direkt an den Slave weitergegeben werden. Ein Power-On-Reset-Generator sorgt für einen definierten Reset-Zustand nach FPGA-Neustart.

Der `addr_master` erzeugt die Steuerungsimpulse für den Slave. Er durchläuft die Adressen 000 bis 111 und schaltet jeweils eine LED ein, wartet dann eine definierte Verzögerungszeit, schaltet ab und geht zur nächsten Adresse.

```

1 entity addr_master is
2     generic (
3         variable_delay : integer := 12500000
4     );
5     port(
6         clk : in std_logic;
7         rst : in std_logic;
8         adr : out std_logic_vector(2 downto 0);

```

```

9         we : out std_logic;
10        d : out std_logic
11    );
12 end entity;
13
14 architecture addr_master_a of addr_master is
15     type state_type is (START, LON, LOFF);
16     signal state : state_type := START;
17     signal cnt : integer range 0 to 12500000 := 0;
18     signal adr_reg : unsigned(2 downto 0) := "000";
19     signal delay_cnt : integer range 0 to variable_delay := 0;
20     signal slave_we : std_logic := '0';
21     signal slave_d : std_logic := '0';
22     signal delay_cnt2 : integer range 0 to 20000 := 0;
23     signal wait_done1 : std_logic := '0';
24     signal wait_done2 : std_logic := '0';
25 begin
26
27     process (clk, rst)
28     begin
29
30         if rst = '1' then
31             state <= START;
32             slave_we <= '0';
33             slave_d <= '0';
34             wait_done1 <= '0';
35             wait_done2 <= '0';
36         elsif rising_edge(clk) then
37             case state is
38                 when START =>
39                     state <= LON;
40                     wait_done1 <= '0';
41                     wait_done2 <= '0';
42                 when LON =>
43                     slave_d <= '1';
44                     if delay_cnt = 0 then
45                         slave_we <= '1';
46                         delay_cnt <= delay_cnt + 1;
47                     elsif delay_cnt = 10 then
48                         slave_we <= '0';
49                         delay_cnt <= delay_cnt + 1;
50                     elsif delay_cnt = variable_delay then
51                         delay_cnt <= 0;
52                         wait_done1 <= '1';
53                     else
54                         delay_cnt <= delay_cnt + 1;
55                     end if;
56
57                     if wait_done1 = '1' then
58                         state <= LOFF;

```

```

59         wait_done1 <= '0';
60     end if;
61     when LOFF =>
62         slave_d <= '0';
63         slave_we <= '1';
64
65         if delay_cnt2 = 10 then
66             delay_cnt2 <= 0;
67             slave_we <= '0';
68             wait_done2 <= '1';
69         else
70             delay_cnt2 <= delay_cnt2 + 1;
71         end if;
72
73         if wait_done2 = '1' then
74             if adr_reg = "111" then
75                 adr_reg <= "000";
76             else
77                 adr_reg <= adr_reg + 1;
78             end if;
79
80             state <= LON;
81             wait_done2 <= '0';
82         end if;
83     end case;
84 end if;
85 end process;
86
87 adr <= std_logic_vector(adr_reg);
88 d <= slave_d;
89 we <= slave_we;
90 end architecture addr_master_a;

```

Der Master verwendet einen Zustandsautomaten mit den Zuständen:

- **START**
- **LON** (LED einschalten)
- **LOFF** (LED ausschalten)

Das folgende Zustandsdiagramm zeigt die Übergänge zwischen den Zuständen.

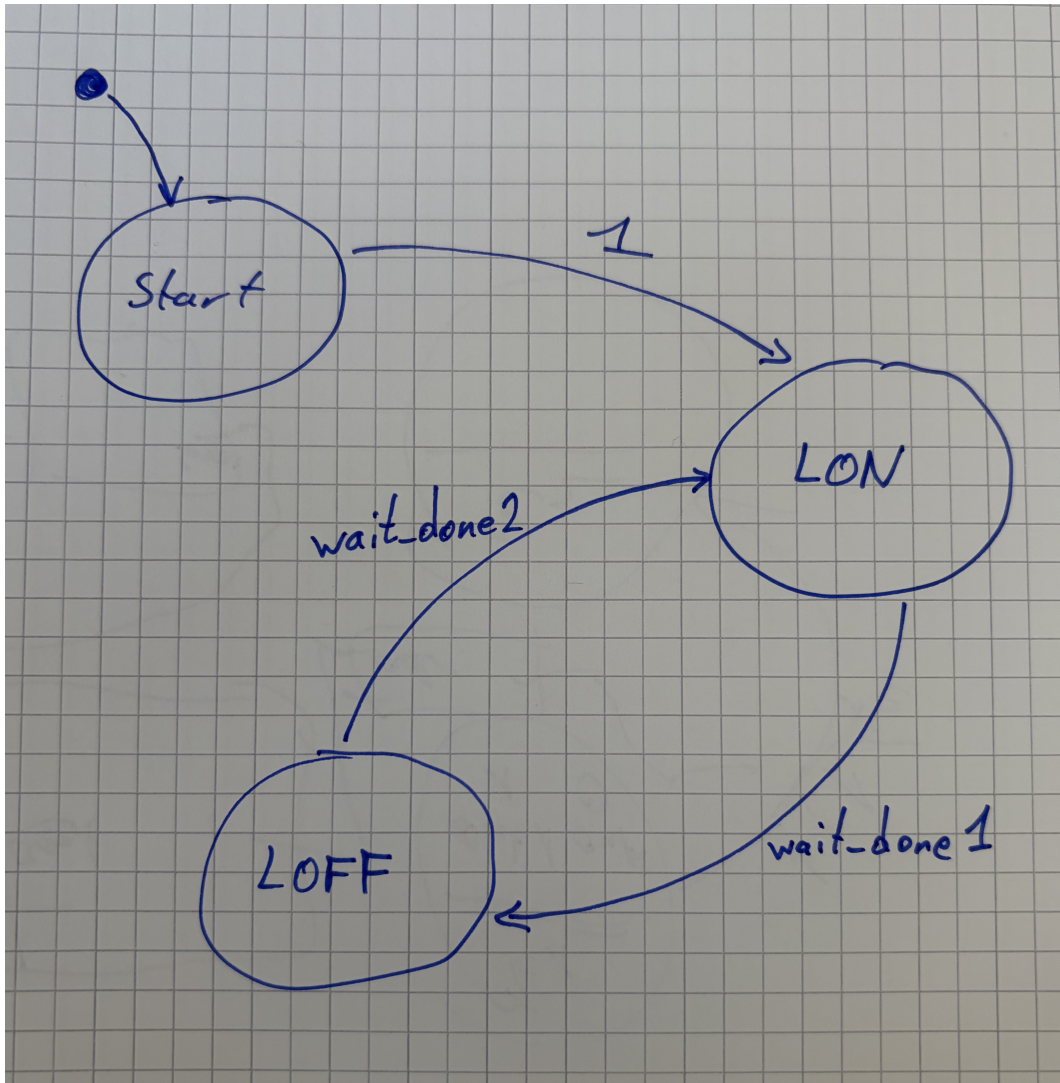


Abbildung 3: Zustandsdiagramm des Address-Master

1.6 Testbench

Die Testbench `addressing_tb.vhd` implementiert die Addressing-Simulation mit verkürztem Delay.

```
1  entity addressing_tb is
2      generic (
3          variable_delay : integer := 500
4      );
5
6      port(
7          clk_12MHz : in std_logic;
8          clk_50MHz : in std_logic;
9          leds : out std_logic_vector(7 downto 0)
10     );
11 end entity;
12
13 architecture addressing_tb_a of addressing_tb is
14     signal async_reset : std_logic := '1';
15     signal slave_we : std_logic := '0';
16     signal slave_d : std_logic := '0';
17     signal slave_addr : std_logic_vector(2 downto 0) := "000";
18     signal clk_12MHz_sim : std_logic := '0';
19     signal clk_50MHz_sim : std_logic := '0';
20     signal counter : unsigned(7 downto 0) := (others => '0');
21     begin
22
23         clk_12MHz_sim <= not clk_12MHz_sim after 41.666666 ns;
24         clk_50MHz_sim <= not clk_50MHz_sim after 10 ns;
25
26         process(clk_12MHz_sim) is
27             variable cnt : unsigned(7 downto 0) := (others => '0');
28             begin
29                 if rising_edge(clk_12MHz_sim) then
30                     if cnt < 255 then
31                         cnt := cnt + 1;
32                         counter <= cnt;
33                         async_reset <= '1';
34                     else
35                         counter <= cnt;
36                         async_reset <= '0';
37                     end if;
38                 end if;
39             end process;
40
41     addr_master_instance : entity work.addr_master
42     generic map(
43         variable_delay => variable_delay
44     )
45     port map(
```

```

46         clk => clk_50MHz_sim,
47         rst => async_reset,
48         adr => slave_addr,
49         we => slave_we,
50         d => slave_d
51     );
52
53     addr_slave_instance : entity work.addr_slave
54     port map(
55         clk => clk_12MHz_sim,
56         we => slave_we,
57         d => slave_d,
58         rst => async_reset,
59         addr => slave_addr,
60         leds => leds
61     );
62
63
64     end architecture addressing_tb_a;

```

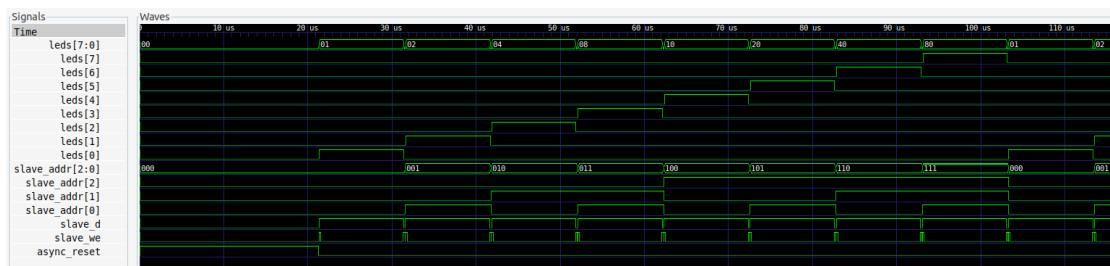


Abbildung 4: Simulation des Address-Master mit Slave

Die Simulation zeigt die korrekte Funktionalität des Masters und Slaves. Der Master durchläuft die Adressen von '000' bis '111', schaltet die entsprechenden LEDs ein und aus.

1.7 Metastabilität

Flip-Flops können metastabil werden, wenn asynchrone Eingangssignale sehr nahe an der aktiven Flanke des Taktes wechseln. In diesem Zustand bleibt das Flip-Flop für eine unbestimmte Zeit in einem undefinierten Spannungsbereich, was zu Glitches, Fehlern oder falschen Datenweiterleitungen führen kann.

1.7.1 Ursachen

- asynchrone Signaländerungen nahe am Taktflankenzeitpunkt
- nicht eingehaltene Setup- und Hold-Zeiten
- fehlende Synchronisation zwischen verschiedenen Taktdomänen

1.7.2 Auswirkungen

- fehlerhafte Signalausgabe
- unvorhersehbares Verhalten von State Machines
- sporadische Systemausfälle

1.7.3 Vermeidung

- Synchronisationsstufen (z. B. zwei Flip-Flops hintereinander)
- ausreichende Setup- und Hold-Zeiten
- Verwendung von Taktdomänen-Grenzen und Cross-Clock-Handshake

2 UART TX

Das folgende Blockdiagramm zeigt die Struktur des UART TX Moduls. Es besteht aus einem Hauptmodul `uart_tx` und einem Testbench-Modul `uart_tx_tb`. Das Hauptmodul implementiert die UART TX Funktionalität, während das Testbench-Modul zur Simulation und Verifikation der Funktionalität des Hauptmoduls dient. Der `clk_ext`-Eingang zeigt den tatsächlichen Clock-Eingang, falls eine externe Clock verwendet werden würde. Da wir jedoch die simulierte Clock der Testbench verwenden, ist dieser Eingang nur symbolisch. Die Signale `rst`, `clk 12MHz`, `byte_in` und `we` werden von der Testbench erzeugt, um die Funktionalität des UART TX Moduls zu testen.

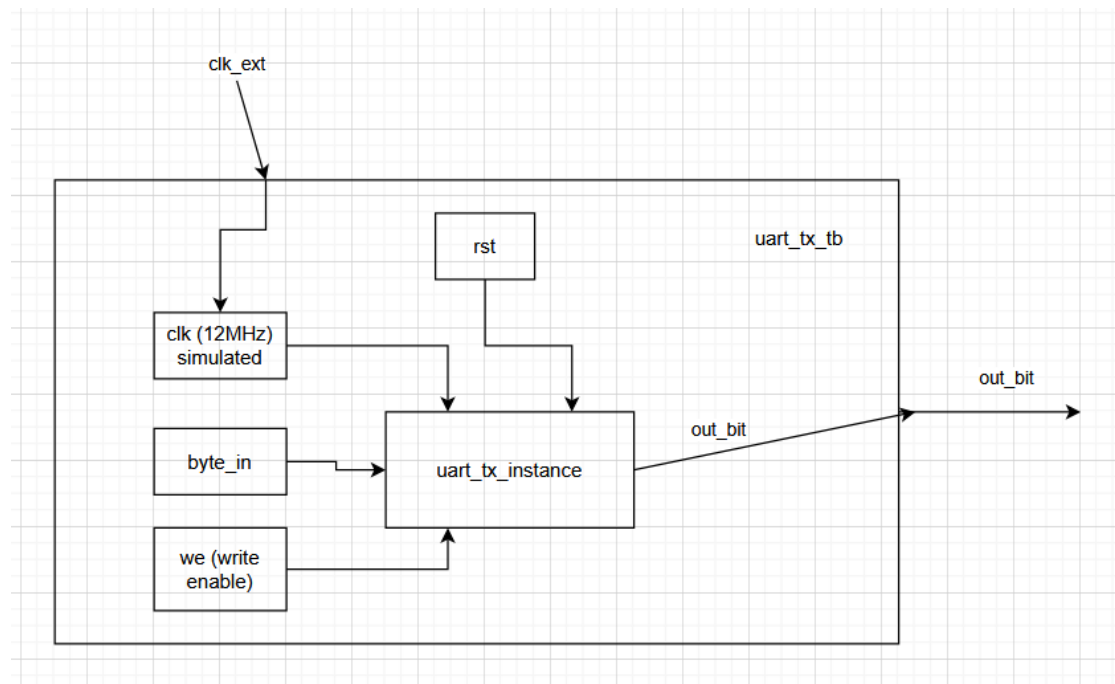


Abbildung 5: UART TX Blockdiagramm

Die Statemaschine des UART TX Moduls besteht aus zwei Hauptzuständen: IDLE und SENDING. Der Automat ist deterministisch.

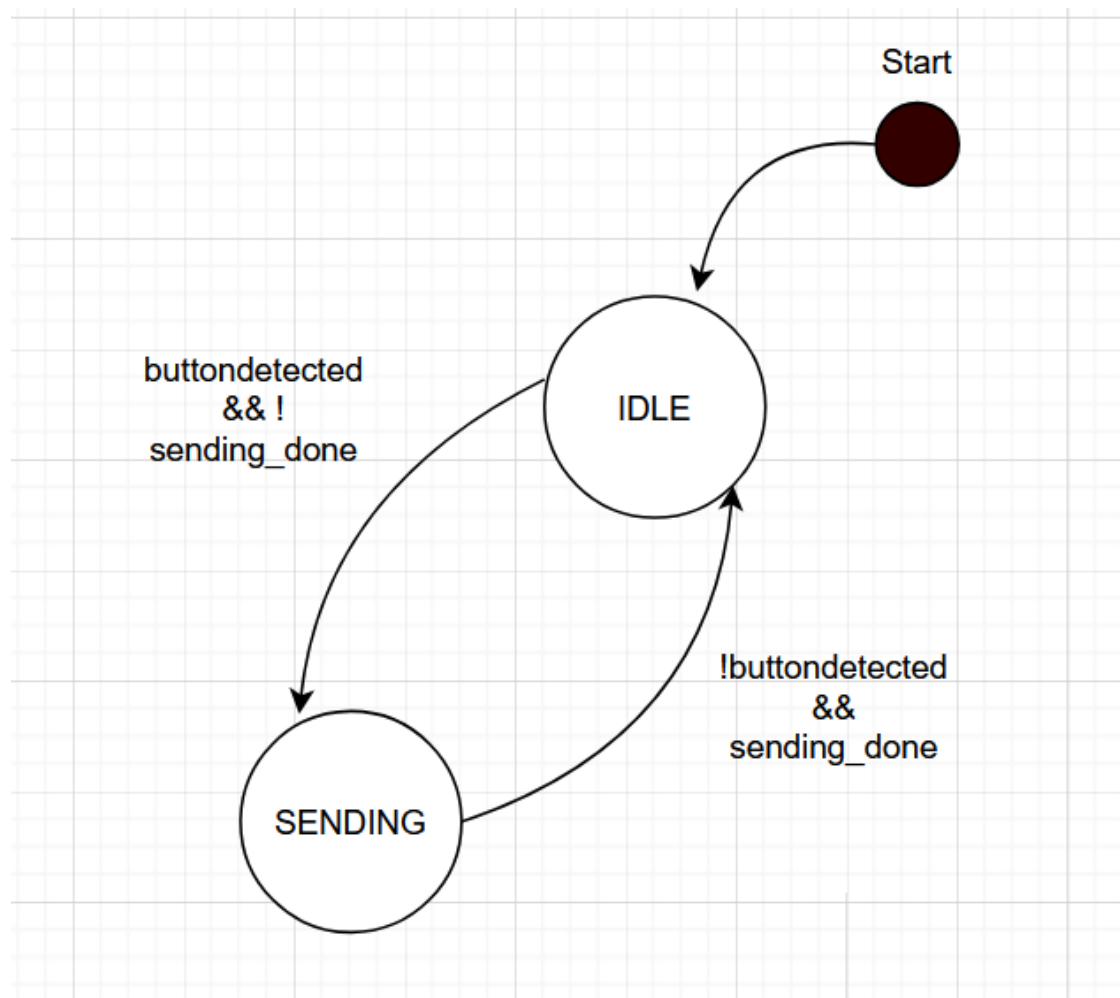


Abbildung 6: UART TX Statemaschine

```

1  library ieee;
2  use IEEE.std_logic_1164.all;
3  use IEEE.numeric_std.all;
4
5  entity uart_tx is
6      port(
7          clk : in std_logic;
8          byte_in : in std_logic_vector(7 downto 0);
9          we : in std_logic;
10         out_bit : out std_logic;
11         rst : in std_logic
12     );
13     end entity;
14
15
16 architecture uart_tx_a of uart_tx is
17     type state_type is (IDLE, SENDING);
18     signal state : state_type := IDLE;
19     signal symbol_cnt : integer := 0;
20     constant baud_rate : integer := 9600;
21     constant clk_speed : integer := 12000000;
22     constant bit_period : integer := clk_speed / baud_rate;
23     signal sending_done : std_logic := '0';
24     signal button_detected : std_logic := '0';
25
26 begin
27     process(clk) is
28         variable cnt_period : integer := 0;
29         variable start_bit_done : std_logic := '0';
30         variable char_bits_done : std_logic := '0';
31     begin
32         if rising_edge(clk) then
33
34             if rst = '1' then
35                 state <= IDLE;
36                 out_bit <= '1';
37             else
38
39                 if button_detected = '1' then
40                     state <= SENDING;
41                     button_detected <= '0';
42                 elsif sending_done = '1' then
43                     state <= IDLE;
44                     sending_done <= '0';
45                 end if;
46
47                 case state is
48                     when IDLE =>
49                         if we = '1' then
50                             button_detected <= '1';

```

```

51         else
52             out_bit <= '1';
53         end if;
54         when SENDING =>
55             if start_bit_done = '0' then
56                 out_bit <= '0';
57                 if cnt_period < bit_period then
58                     cnt_period := cnt_period + 1;
59                 else
60                     cnt_period := 0;
61                     start_bit_done := '1';
62                 end if;
63             elsif char_bits_done = '1' then
64                 out_bit <= '1';
65                 if cnt_period < bit_period then
66                     cnt_period := cnt_period + 1;
67                 else
68                     cnt_period := 0;
69                     char_bits_done := '0';
70                     start_bit_done := '0';
71                     sending_done <= '1';
72                 end if;
73             else
74                 if symbol_cnt < 8 then
75
76                     out_bit <= byte_in(symbol_cnt);
77                     cnt_period := cnt_period + 1;
78
79                     if cnt_period > (bit_period - 1 ) then
80                         symbol_cnt <= symbol_cnt + 1;
81                         cnt_period := 0;
82                     end if;
83                 else
84                     symbol_cnt <= 0;
85                     char_bits_done := '1';
86                 end if;
87             end if;
88
89         end case;
90
91     end if;
92
93     end if;
94 end process;
95 end architecture uart_tx_a;

```

Vor dem Case-Statement wird geprüft ob eine der Zustandsübergangskriterien erfüllt ist. Wenn der we-Eingang aktiv ist, wird das Zustandsübergangssignal button_detected auf '1' gesetzt, wodurch der Übergang von IDLE zu SENDING ausgelöst wird. Sobald

das Senden abgeschlossen ist, wird das Zustandsübergangssignal `sending_done` auf '1' gesetzt, wodurch der Übergang von SENDING zu IDLE ausgelöst wird.

```

1  library ieee;
2  use IEEE.std_logic_1164.all;
3  use IEEE.numeric_std.all;
4
5  entity uart_tx_tb is
6  end entity;
7
8
9  architecture uart_tx_tb_a of uart_tx_tb is
10     signal clk : std_logic := '0';
11     signal byte_in : std_logic_vector(7 downto 0) := (others => '0');
12     signal we : std_logic := '0';
13     signal rst : std_logic := '0';
14     signal out_bit : std_logic := '0';
15
16 begin
17     uart_tx_inst : entity work.uart_tx
18     port map(
19         clk=> clk,
20         byte_in => byte_in,
21         we => we,
22         rst => rst,
23         out_bit => out_bit
24     );
25
26     clk <= not clk after 83.33 ns / 2;
27
28     process
29     begin
30         -- Reset
31         rst <= '1';
32         wait for 100 ns;
33         rst <= '0';
34
35         -- Check IDLE
36         wait for 50 ns;
37         assert out_bit = '1' report "ERROR: out_bit not idle high after reset"
38             <- severity error;
39
40         -- 1
41         byte_in <= "11010001";
42         wait for 100 ns;
43         we <= '1';
44         wait for 100 ns;
45         we <= '0';
46
47         wait for 50 us;
48         assert out_bit = '0'
49             report "ERROR: Startbit not detected (Test 1)"
50                 severity warning;

```

```

50
51     wait for 2 ms;
52     assert out_bit = '1'
53         report "ERROR: Line not back to idle (Test 1)"
54             severity error;
55
56     -- 2
57     byte_in <= "10101010";
58     wait for 100 ns;
59     we <= '1';
60     wait for 300 ns;
61     we <= '0';
62
63     wait for 50 us;
64     assert out_bit = '0'
65         report "ERROR: Startbit not detected (Test 2)"
66             severity warning;
67
68     wait for 2 ms;
69
70     -- 3
71     byte_in <= "11110000";
72     wait for 100 ns;
73     we <= '1';
74     wait for 300 ns;
75     we <= '0';
76
77     wait for 50 us;
78     assert out_bit = '0'
79         report "ERROR: Startbit not detected (Test 3)"
80             severity warning;
81
82     wait for 2 ms;
83
84     -- 4
85     byte_in <= "00001111";
86     wait for 100 ns;
87     we <= '1';
88     wait for 300 ns;
89     we <= '0';
90
91     wait for 50 us;
92     assert out_bit = '0'
93         report "ERROR: Startbit not detected (Test 4)"
94             severity warning;
95
96     wait for 2 ms;
97
98     -- 5
99     byte_in <= "00110011";

```



```

100     wait for 100 ns;
101     we <= '1';
102     wait for 300 ns;
103     we <= '0';
104
105     wait for 50 us;
106     assert out_bit = '0'
107         report "ERROR: Startbit not detected (Test 5)"
108             severity warning;
109
110     wait for 2 ms;
111
112     -- Ende
113     assert false report "Simulation finished successfully" severity note;
114     wait;
115
116
117     end process;
118 end architecture uart_tx_tb_a;

```

Die Testbench testet fünf mal die Funktionsalität des TX Moduls. Es werden jeweils verschiedene Byte-Werte gesendet. Nach jedem Senden wird überprüft, ob das Startbit korrekt erkannt wurde und ob die Leitung nach dem Senden wieder auf den Idle-Zustand zurückkehrt. Die Asserts dienen zur Fehlererkennung. Schlägt keines der Asserts fehl, so wird am Ende der Simulation eine Erfolgsmeldung ausgegeben. Die folgenden drei Bilder zeigen drei der Tests aus der Simulation. Zu sehen ist, dass die Variable `symbol_cnt` von 0 bis 7 hochzählt und, dabei die entsprechenden Bits des `byte_in`-Signals auf das `out_bit`-Signal überträgt. Auch die Funktionalität der Start- und Stoppbits ist gut zu erkennen.

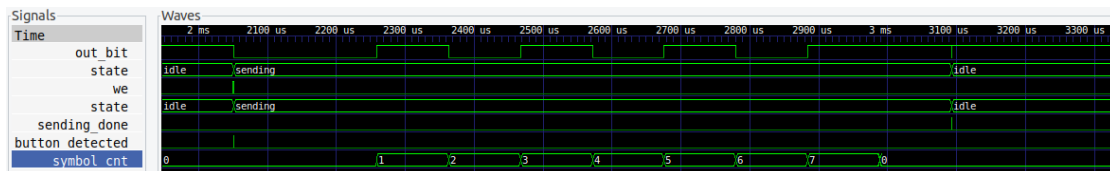


Abbildung 7: UART TX Simulation 1

Testcase 2 zeigt die Übertragung des Byte-Werts "10101010".

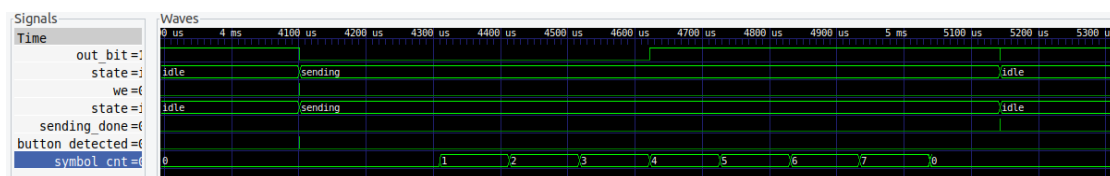


Abbildung 8: UART TX Simulation 2

Testcase 3 zeigt die Übertragung des Byte-Werts "11110000".

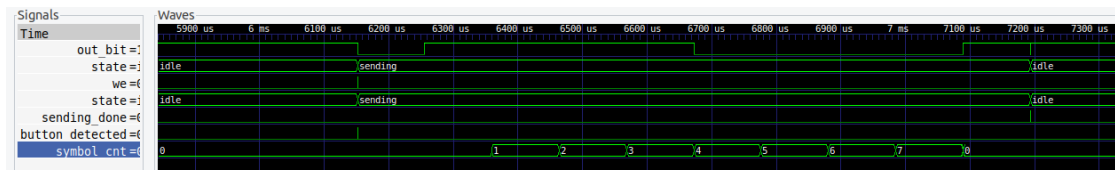


Abbildung 9: UART TX Simulation 3

Testcase 4 zeigt die Übertragung des Byte-Werts "00001111".

3 Aufgabe 4: UART RX

Das folgende Blockdiagramm zeigt die Struktur des UART RX Moduls. Es besteht aus einem Hauptmodul `uart_rx` und einem Testbench-Modul `uart_rx_tb`. Das Hauptmodul implementiert die UART RX Funktionalität: Es empfängt serielle Daten über den `uart_in`-Eingang, synchronisiert das eingehende Signal, sampled die Datenbits in der Mitte des Bitfensters und schaltet die LED je nach empfangenem Zeichen ein oder aus. Das Testbench-Modul dient zur Simulation und Verifikation der Funktionalität des Hauptmoduls.

Das Modul arbeitet mit einer Baudrate von 9600 bei einem 12 MHz Systemtakt (8N1-Konfiguration: 8 Datenbits, kein Paritätsbit, 1 Stoppbit). Empfängt das Modul das Zeichen 'j' (ASCII 106, 0x6A), wird die LED eingeschaltet. Empfängt es das Zeichen 'k' (ASCII 107, 0x6B), wird die LED ausgeschaltet. Alle anderen empfangenen Zeichen werden ignoriert und der LED-Zustand bleibt unverändert.

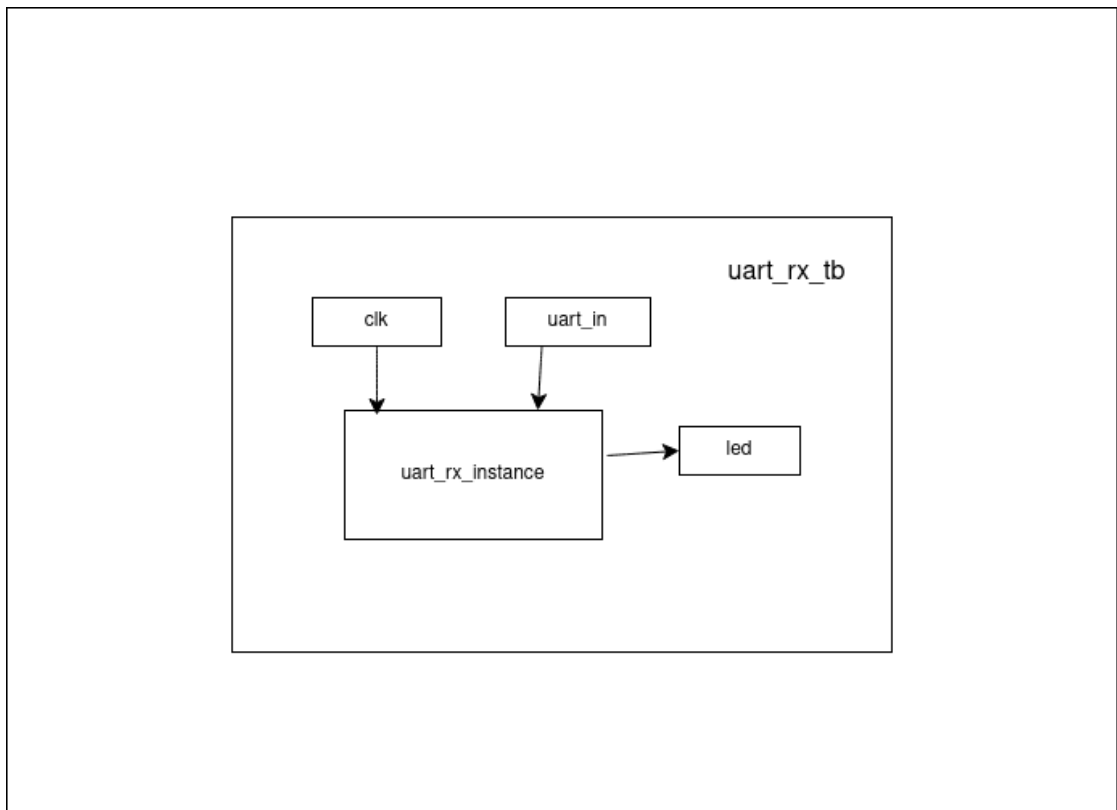


Abbildung 10: UART RX Blockdiagramm

Die Statemaschine des UART RX Moduls besteht aus zwei Hauptzuständen: `IDLE` und `READING`. Der Automat ist deterministisch.

Im Zustand `IDLE` wartet das Modul auf eine fallende Flanke am `uart_in`-Signal, die das Startbit signalisiert. Durch die Zweiflanken-Synchronisation (`uart_sync`) werden metastabile Zustände vermieden. Bei erkannter fallender Flanke ("10") wechselt der Automat in den Zustand `READING` und initialisiert alle Zähler.

Im Zustand `READING` wartet das Modul zunächst eine halbe Bitperiode (`pre_cnt`), um das Sampling in die Mitte des ersten Datenbits zu verschieben. Anschließend werden 8 Datenbits eingelesen, wobei jeweils nach einer vollen Bitperiode (`clk_cnt`) gesampelt wird. Nach dem letzten Datenbit wird noch eine halbe Bitperiode gewartet (`post_cnt`), um das Stoppbit zu überspringen. Danach wird der empfangene Wert im `cache` mit den ASCII-Codes für 'j' und 'k' verglichen und die LED entsprechend geschaltet.

```

1  library ieee;
2  use IEEE.std_logic_1164.all;
3  use IEEE.numeric_std.all;
4
5  entity uart_rx is
6      port(
7          clk : in std_logic;
8          uart_in : in std_logic;
9          led : out std_logic
10     );
11     end entity;
12
13 architecture uart_rx_arch of uart_rx is
14
15     signal cache : std_logic_vector(7 downto 0) := (others => '0');
16     signal led_signal : std_logic := '0';
17     signal uart_sync : std_logic_vector(1 downto 0) := (others => '1');
18     constant baud_rate : integer := 9600;
19     constant clk_freq : integer := 12000000;
20     constant bit_period : integer := clk_freq / baud_rate - 1;
21     constant post_cnt_max : integer := (3 * bit_period) / 2;
22     type state_type is (IDLE, READING);
23     signal state : state_type := IDLE;
24
25 begin
26
27     process(clk)
28         variable pre_cnt : integer range 0 to bit_period / 2 := 0;
29         variable post_cnt : integer range 0 to (3 * bit_period) / 2 := 0;
30         variable bit_cnt : integer range 0 to 8 := 0;
31         variable clk_cnt : integer range 0 to bit_period := 0;
32
33     begin
34         if rising_edge(clk) then
35
36             uart_sync <= uart_sync(0) & uart_in; -- Shift in the new bit for
37             ↪ synchronization
38
39             if state = IDLE then
40
41                 if uart_sync(1 downto 0) = "10" then -- Detect the falling edge
42                     ↪ for start bit
43                     state <= READING;
44                     pre_cnt := 0;
45                     post_cnt := 0;
46                     bit_cnt := 0;
47                     clk_cnt := 0;
48                     cache <= (others => '0');
49                 end if;

```

```

49     elsif state = READING then
50
51         if pre_cnt < (bit_period / 2) then
52
53             pre_cnt := pre_cnt + 1;
54
55         elsif bit_cnt < 8 then
56
57             if clk_cnt < bit_period then
58
59                 clk_cnt := clk_cnt + 1;
60
61             else
62
63                 cache(bit_cnt) <= uart_sync(0); -- Sample the bit
64                 bit_cnt := bit_cnt + 1;
65                 clk_cnt := 0;
66
67             end if;
68
69         elsif post_cnt < (post_cnt_max) then
70
71             post_cnt := post_cnt + 1;
72
73         else
74
75             if unsigned(cache) = 106 then --checks if the received
76                 ⇨ byte is 106 (ASCII for 'j')
77                 led_signal <= '1';
78             elsif unsigned(cache) = 107 then --checks if the received
79                 ⇨ byte is 107 (ASCII for 'k')
80                 led_signal <= '0';
81             end if;
82             state <= IDLE;
83
84         end if;
85     end if;
86 end process;
87
88 led <= led_signal;
89
90 end architecture uart_rx_arch;

```

Der VHDL-Code implementiert den beschriebenen Empfänger. Besonders wichtig sind:

- **Zweiflanken-Synchronisation** (Zeile 30): `uart_sync <= uart_sync(0) & uart_in` verhindert Metastabilität durch asynchrone Eingangssignale.
- **Startbiterkennung** (Zeile 34): Die fallende Flanke "10" am synchronisierten

Signal löst den Empfang aus.

- **Bit-Sampling in der Mitte des Fensters:** `pre_cnt` verschiebt das erste Sampling um eine halbe Bitperiode, `clk_cnt` sorgt für die korrekte Timing der folgenden Bits.
- **Zeichenerkennung** (Zeile 56–59): Nach vollständigem Empfang wird der `cache` mit 106 (`'j'`) und 107 (`'k'`) verglichen.

```

1  library ieee;
2  use IEEE.std_logic_1164.all;
3  use IEEE.numeric_std.all;
4  use std.env.all;
5
6  entity uart_rx_tb is
7  end entity;
8
9  architecture uart_rx_tb_arch of uart_rx_tb is
10
11     signal clk : std_logic := '0';
12     signal uart_in : std_logic := '1';
13     signal led : std_logic;
14
15     constant baud_rate : integer := 9600;
16     constant clk_freq : integer := 12000000;
17     constant clk_periode : time := 83.33 ns;
18     constant baud_period : time := 1 sec / baud_rate;
19
20 begin
21
22     uart_rx_inst : entity work.uart_rx
23     port map(
24         clk => clk,
25         uart_in => uart_in,
26         led => led
27     );
28
29
30     clk <= not clk after clk_periode/2; -- 12 MHz clock
31
32     process
33         variable byte_jump : std_logic_vector(7 downto 0) := "01101010"; --
34         ↪ ASCII for 'j'
35         variable byte_kill : std_logic_vector(7 downto 0) := "01101011"; --
36         ↪ ASCII for 'k'
37         variable byte_other : std_logic_vector(7 downto 0) := "11111111"; -- A
38         ↪ byte that is not 'j' or 'k'
39     begin
40         wait for 5000 ns; -- Wait for the system to stabilize
41
42         -- Send start bit
43         uart_in <= '0';
44         wait for baud_period; -- Wait for one bit period (1/baud_rate)
45
46         -- Send data bits (LSB first)
47         for i in 0 to 7 loop
48             uart_in <= byte_jump(i);
49             wait for baud_period; -- Wait for one bit period
50         end loop;

```



```

48
49      -- Send stop bit
50      uart_in <= '1';
51      wait for baud_period; -- Wait for one bit period
52
53      wait for 5000 ns; -- Wait before ending the simulation
54
55      assert led = '1' report "LED should be ON after receiving 'j'" severity
56      ↪ error;
57
58      wait for 500000 ns; -- Wait before sending the next byte
59
60      -- Send byte for 'k'
61      uart_in <= '0';
62      wait for baud_period; -- Wait for one bit period (1/baud_rate)
63
64      -- Send data bits (LSB first)
65      for i in 0 to 7 loop
66          uart_in <= byte_kill(i);
67          wait for baud_period; -- Wait for one bit period
68      end loop;
69
70      -- Send stop bit
71      uart_in <= '1';
72
73      wait for baud_period; -- Wait for one bit period
74
75      wait for 5000 ns; -- Wait before ending the simulation
76
77      assert led = '0' report "LED should be OFF after receiving 'k'"
78      ↪ severity error;
79
80      wait for 500000 ns; -- Wait before sending the next byte
81
82      -- test with a different byte (not 'j' or 'k')
83      uart_in <= '0';
84
85      wait for baud_period; -- Wait for one bit period (1/baud_rate)
86
87      -- Send data bits (LSB first)
88      for i in 0 to 7 loop
89          uart_in <= byte_other(i); -- Send a byte that is not 'j' or 'k'
90          wait for baud_period; -- Wait for one bit period
91      end loop;
92
93      -- Send stop bit
94      uart_in <= '1';
95
96      wait for baud_period; -- Wait for one bit period

```

```

96     wait for 50000 ns; --
97
98     assert led = '0' report "LED should remain OFF after receiving a byte
    ↪ other than 'j' or 'k'" severity error;
99
100    report "Simulation finished successfully";
101    stop(0); -- Clean exit, no error code
102    end process;
103
104 end architecture;

```

Die Testbench verifiziert die korrekte Funktionalität des UART RX Moduls mit drei Testfällen:

1. **Testfall 1 – LED einschalten:** Es wird das Zeichen 'j' (ASCII 106, 01101010) gesendet. Das Startbit (0), gefolgt von den 8 Datenbits (LSB first) und dem Stoppbit (1) werden seriell über `uart_in` übertragen. Nach Abschluss des Empfangs wird mit einem Assert geprüft, ob `led = '1'` ist.
2. **Testfall 2 – LED ausschalten:** Es wird das Zeichen 'k' (ASCII 107, 01101011) gesendet. Der Ablauf ist identisch zu Testfall 1. Das Assert prüft, ob `led = '0'` ist.
3. **Testfall 3 – Ungültiges Zeichen:** Es wird das Zeichen 0xFF (11111111) gesendet, das weder 'j' noch 'k' entspricht. Das Assert prüft, dass die LED ihren vorherigen Zustand ('0') beibehält.

Alle Testfälle verwenden die korrekte Baudrate von 9600 Bd (Bitperiode $\sim 104,17 \mu\text{s}$) und den 12 MHz Systemtakt. Die Asserts dienen zur automatisierten Fehlererkennung: Schlägt keines fehl, so ist die Grundfunktionalität des Empfängers verifiziert.



Abbildung 11: GTKWave-Simulation des UART RX Moduls: Verlauf der Signale `state`, `uart_in` und `led`

Die Simulation zeigt den vollständigen Ablauf der drei Testfälle. Von oben nach unten sind die Signale `state` (Zustandsautomat), `uart_in` (serieller Dateneingang) und `led` (Ausgang) dargestellt.

Im Verlauf von `uart_in` sind die drei Übertragungen erkennbar: Zunächst das Zeichen 'j' (Startbit, gefolgt von 01101010, Stoppbit), anschließend 'k' (01101011) und zuletzt 0xFF (11111111). Das `state`-Signal wechselt jeweils von IDLE zu READING während des Empfangs und zurück zu IDLE nach Abschluss. Das `led`-Signal zeigt den erwarteten Verlauf: Es geht nach Empfang von 'j' auf '1', fällt nach Empfang von 'k' auf '0' und bleibt bei ungültigem Zeichen unverändert.

4 Aufgabe 5.2: Adressierungsarten und Alignment

Die Aufgabe untersucht, wie der RISC-V Cross-Compiler unterschiedliche Speicherzugriffe auf ein `uint8_t`-Array übersetzt, wenn dieses über einen `uint32_t`-Pointer gelesen wird. Dabei wird das Phänomen des *Data Structure Alignment* anhand des erzeugten Assemblercodes analysiert.

Das C-Programm definiert ein Byte-Array `Values[120]` und liest aus drei verschiedenen Positionen jeweils ein 32-Bit-Wort:

```

1  uint8_t  Values[COUNT_VALUES];
2  uint32_t *value_ad1, *value_ad2, *value_ad3;
3  uint32_t value1, value2, value3;
4
5  value_ad1 = (uint32_t*)&(Values[0]);           // Zeile 11
6  value_ad2 = (uint32_t*)&(Values[8+2]);         // Zeile 12
7  value_ad3 = (uint32_t*)&(Values[12+1]);        // Zeile 13
8
9  value1 = *(value_ad1);                         // Zeile 14
10 value2 = *(value_ad2);                         // Zeile 15
11 value3 = *(value_ad3);                         // Zeile 16

```

Die Adressen der drei Zugriffe weisen unterschiedliche Alignments auf:

Variable	Offset	Alignment (mod 4)	Status
Values[0]	0	0	4-Byte-aligned
Values[10]	10	2	nicht 4-Byte-aligned
Values[13]	13	1	nicht 4-Byte-aligned

4.1 Assemblercode der drei Zuweisungen

Aus dem Disassembly (main.asm) wurden die Instruktionen extrahiert, die den Zeilen 14, 15 und 16 des C-Codes entsprechen. Die Basisadresse des Arrays `Values` liegt im Register `a5` (0x80000018).

Zeile 14: `value1 = *(value_ad1);` – Aligned-Zugriff (Offset 0)

```
1 164: 0007a683    lw a3, 0(a5)    # Load Word (32 Bit): value1 = *(Values+0),
    ↪ aligned
2 168: 80000737    lui a4, 0x80000 # Lade obere 20 Bit der Zieladresse
3 16c: 00d72023    sw a3, 0(a4)    # Speichere value1 an Adresse 0x80000000
```

Der Zugriff auf `Values[0]` ist an einer durch 4 teilbaren Adresse. Der Compiler verwendet einen einzigen `lw`-Befehl (*Load Word*), der das komplette 32-Bit-Wort in einem Taktzyklus über den 32-Bit-Wishbone-Bus lädt.

Zeile 15: `value2 = *(value_ad2);` – Unaligned-Zugriff (Offset 10)

```
1 170: 00c7d703    lhu a4, 12(a5)  # Load Half Unsigned: untere 16 Bit (Bytes
    ↪ 12,13)
2 174: 00a7d683    lhu a3, 10(a5)  # Load Half Unsigned: obere 16 Bit (Bytes
    ↪ 10,11)
3 178: 01071713    slli a4, a4, 16 # Schiebe Bytes 12,13 an Bit-Position 16..31
4 17c: 00d76733    or a4, a4, a3   # Kombiniere zu 32-Bit-Wort: value2 =
    ↪ Bytes[10..13]
5 180: 80e1a223    sw a4, -2044(gp) # Speichere value2 an globale Variable
```

Der Zugriff auf `Values[10]` ist nicht 4-Byte-aligned ($10 \bmod 4 = 2$). Ein `lw` würde einen Alignment-Trap auslösen. Der Compiler lädt deshalb zwei 16-Bit-Halbwörter (`lhu`) und setzt diese per `slli` (Shift) und `or` zu einem 32-Bit-Wert zusammen.

Zeile 16: `value3 = *(value_ad3);` – Unaligned-Zugriff (Offset 13)

```
1 184: 00e7c703    lbu a4, 14(a5)  # Load Byte Unsigned: Byte 14
2 188: 00d7c683    lbu a3, 13(a5)  # Load Byte Unsigned: Byte 13
3 18c: 00871713    slli a4, a4, 8  # Schiebe Byte 14 an Position 8..15
4 190: 00d766b3    or a3, a4, a3   # Kombiniere Bytes 13,14
5 194: 00f7c703    lbu a4, 15(a5)  # Load Byte Unsigned: Byte 15
6 198: 0107c783    lbu a5, 16(a5)  # Load Byte Unsigned: Byte 16
7 19c: 01071713    slli a4, a4, 16 # Schiebe Byte 15 an Position 16..23
8 1a0: 00d76733    or a4, a4, a3   # Kombiniere Bytes 13..15
9 1a4: 01879793    slli a5, a5, 24 # Schiebe Byte 16 an Position 24..31
10 1a8: 00e7e7b3    or a5, a5, a4   # Komplettes 32-Bit-Wort: value3 =
    ↪ Bytes[13..16]
11 1ac: 80f1a423    sw a5, -2040(gp) # Speichere value3 an globale Variable
```

Der Zugriff auf `Values[13]` ist völlig unaligned ($13 \bmod 4 = 1$). Der Compiler muss vier einzelne Bytes (`lbu`) laden und durch drei Shift-Operationen (`slli`) sowie drei OR-Operationen (`or`) manuell zu einem 32-Bit-Wort zusammensetzen.

4.2 Analyse: Unterschiede und Erklärung

Die drei scheinbar identischen C-Zuweisungen `valueX = *(pointer)` werden vom Compiler in fundamental unterschiedliche Assembler-Sequenzen übersetzt:

Zuweisung	Anzahl Befehle	Zugriffsart
<code>value1</code> (Offset 0)	3	Einzelnes <code>lw</code> (32-Bit aligned)
<code>value2</code> (Offset 10)	5	Zwei <code>lhu</code> + Shift/Or (16-Bit kombiniert)
<code>value3</code> (Offset 13)	10	Vier <code>lbu</code> + Shift/Or (8-Bit kombiniert)

Beobachtung: Der aligned Zugriff (`value1`) ist um den Faktor 3 schneller und benötigt nur ein Drittel des Codespeichers im Vergleich zum völlig unaligned Zugriff (`value3`).

Erklärung – Data Structure Alignment:

Die NEORV32 ist eine 32-Bit-RISC-V-CPU mit einem 32-Bit-Wishbone-Bus. Der Befehlssatz erlaubt einen `lw` (*Load Word*) nur an Adressen, die durch 4 teilbar sind (*4-Byte-Alignment*). Diese Einschränkung resultiert aus der Busarchitektur: Der Speicher liefert in einem Taktzyklus exakt 32 Bit (4 Bytes) an einer Wortgrenze.

Wenn ein `uint32_t`-Pointer auf ein `uint8_t`-Array zeigt, garantiert das C-Standard nicht, dass die Zieladresse für 32-Bit-Zugriffe geeignet ist. Der Datentyp `uint8_t` hat ein Alignment von 1 Byte (er darf an jeder Adresse liegen), während `uint32_t` ein Alignment von 4 Byte erfordert.

Bei unaligned Zugriffen muss der Compiler den 32-Bit-Wert aus mehreren, kleineren, erlaubten Ladungen (`lhu` für 16 Bit, `lbu` für 8 Bit) manuell zusammensetzen. Dies erfordert zusätzliche Schiebe- und Verknüpfungsoperationen, was zu längerer Ausführungszeit und größerem Code führt. Auf Systemen ohne Hardware-Unterstützung für unaligned Zugriffe (wie der NEORV32 im Standard-Konfiguration) würde ein direktes `lw` auf `Values[10]` oder `Values[13]` sogar einen *Alignment-Trip* (Laufzeitfehler) auslösen.

Fazit: Die Wahl der Datenstrukturen und die Berücksichtigung von Alignment-Regeln sind in eingebetteten Systemen mit RISC-V-Architektur entscheidend für Code-Effizienz und Laufzeit. Ein `uint32_t`-Array anstelle des `uint8_t`-Arrays hätte alle Zugriffe aligned gemacht und den Overhead vermieden.

5 Seven-Segment-Anzeige (Aufgabe 6)

5.1 Grundsätzlicher Aufbau

Die Ansteuerung der Sieben-Segment-Anzeige folgt dem im Kurs etablierten Muster der Memory-Mapped-I/O über den Wishbone-Bus. Die Datenflussskette ist wie folgt:

1. Die Software (`main.c`) ruft eine Funktion der Hardware-Abstraction-Layer (HAL) auf
2. Die HAL (`iceduino_sevensegment.c`) schreibt über einen `volatile uint8_t`-Pointer in den Speicher
3. Das VHDL-Top-Level leitet den Buszugriff über den Wishbone-Multiplexer an das `iceduino_sevensegment`-Modul weiter
4. Das Modul speichert das Datenbyte in einem Register und dekodiert es kombinatorisch auf die 7-Segment-Ausgänge

Dieses Vorgehen entspricht der im Script (Kapitel 6) vorgestellten Integration weiterer Peripherie am Beispiel der LED-Ansteuerung. Die Adressierung erfolgt im Bereich `0xF00000xx`, wobei das Seven-Segment-Modul die Basisadresse `0xF0000080` erhält.

5.2 VHDL-Modul: `iceduino_sevensegment.vhd`

Das Modul implementiert einen Wishbone-Slave mit kombinatorischem 7-Segment-Dekoder. Der Schreibzugriff erfolgt taktflankengesteuert, die Segmentdekodierung rein kombinatorisch (keine Taktflanke, keine Register), wie in der Aufgabenstellung gefordert.

Der Dekoder arbeitet auf dem unteren unteren 4 Bits des 8-Bit-Registers, da die Hex-Werte `x"0"` bis `x"9"` nur 4 Bit breit sind und ein Vergleich mit 8-Bit-Vektoren in VHDL eine Fehler erzeugen würde. Bit 7 des Registers ist für die spätere Steuerung des Dezimalpunkts (DP) vorgesehen, wird aktuell jedoch nicht genutzt (`point_status` ist auf '0' initialisiert).

Der Ausgang `sevensegment_o` ist als **Common-Kathode** ausgelegt: Ein Segment leuchtet, wenn der entsprechende Ausgang auf '0' (LOW) liegt. Die Zuordnung der Segmente erfolgt gemäß Datenblatt des Kingbright SC03-12EWA (vgl. Script S. 22):

Tabelle 1: Segmentzuordnung der Ausgänge `sevenssegment_o(7 downto 0)`

Bit-Position	Segment	Logik	Beschreibung
7	a	<code>seg_pattern(0)</code>	Oben (MSB)
6	b	<code>seg_pattern(1)</code>	Oben rechts
5	c	<code>seg_pattern(2)</code>	Unten rechts
4	d	<code>seg_pattern(3)</code>	Unten
3	e	<code>seg_pattern(4)</code>	Unten links
2	f	<code>seg_pattern(5)</code>	Oben links
1	g	<code>seg_pattern(6)</code>	Mittleres Segment
0	DP	<code>not point_status</code>	Dezimalpunkt (LSB)

Die Dekodierung der Ziffern 0–9 ist in Tabelle 2 dargestellt. Die Werte entsprechen der kombinatorischen `when`-Selektion im VHDL-Code.

Tabelle 2: 7-Segment-Dekodierung für Ziffern 0–9 (Common-Kathode, aktiv-LOW)

Ziffer	0	1	2	3	4	5	6	7	8	9
Hex	0x01	0x79	0x24	0x30	0x19	0x12	0x02	0x78	0x00	0x10
a	1	1	0	0	1	0	0	0	0	0
b	0	0	0	0	0	1	1	0	0	0
c	0	0	1	0	0	0	0	0	0	0
d	0	1	0	0	1	0	0	1	0	0
e	0	1	0	1	1	0	0	1	0	1
f	0	1	1	1	1	0	0	1	0	0
g	1	1	0	0	0	0	0	1	0	0
DP	1	1	1	1	1	1	1	1	1	1

Anmerkung: DP = 1 bedeutet *aus* (Common-Kathode, `not point_status` mit `point_status = '0'`)

Listing 1: VHDL-Modul: `iceduino_sevenssegment.vhd` – Wishbone-Slave mit kombinatorischem 7-Segment-Dekoder

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity iceduino_sevenssegment is
5      generic (
6          sevenssegment_addr : std_ulogic_vector(31 downto 0)
7      );
8      port (
9          clk_i   : in  std_ulogic; -- global clock line
10         rstn_i   : in  std_ulogic; -- global reset line, low-active
11         --wishbone-
12         adr_i     : in  std_ulogic_vector(31 downto 0);

```



```

13     dat_i      : in  std_ulogic_vector(31 downto 0); --write to slave
14     dat_o      : out std_ulogic_vector(31 downto 0);
15     we_i       : in  std_ulogic;
16     stb_i      : in  std_ulogic;
17     cyc_i      : in  std_ulogic;
18     ack_o      : out std_ulogic;
19     err_o      : out std_ulogic;
20     -- parallel io --
21     sevensegment_o : out std_ulogic_vector(7 downto 0);
22     oe_enable  : out std_ulogic;
23     oe_enable2 : out std_ulogic
24 );
25 end entity;
26
27 architecture iceduino_sevensegment_rtl of iceduino_sevensegment is
28
29     signal module_active : std_ulogic;
30     signal module_addr   : std_ulogic_vector(31 downto 0);
31     signal reg_sevensegment : std_ulogic_vector(7 downto 0) := (others => '0');
32     signal seg_pattern : std_ulogic_vector(6 downto 0) := (others => '1'); -- initial all
33     -- segments off (common anode)
34     signal point_status : std_ulogic := '0';
35
36 begin
37     -- module active
38     module_active <= '1' when ((adr_i = sevensegment_addr) and (cyc_i = '1' and stb_i =
39     -- '1')) else '0';
40     module_addr   <= adr_i;
41
42     oe_enable <= '1';
43     oe_enable2 <= '1';
44
45     w_access: process(clk_i)
46     begin
47         if rising_edge(clk_i) then
48
49             -- handshake
50             err_o <= '0';
51             if (module_active = '1') then
52                 ack_o <= '1';
53             else
54                 ack_o <= '0';
55             end if;
56             --write
57             if (module_active = '1' and we_i = '1') then
58                 reg_sevensegment <= dat_i(7 downto 0);
59             end if;
60             --read
61             dat_o(31 downto 0) <= (others => '0');
62             if (module_active = '1' and we_i = '0') then
63                 dat_o(7 downto 0) <= reg_sevensegment;
64             end if;
65         end if;
66     end process w_access;

```

```

65
66
67 -- KOMBINATORISCHER 7-SEGMENT-DEKODER
68 -- Common-Anode: Segment leuchtet bei '0' (LOW)
69
70 seg_pattern <=
71 "111110" when reg_sevensegment(3 downto 0) = x"0" else -- 0: a,b,c,d,e,f an, g
    ↪ aus
72 "011000" when reg_sevensegment(3 downto 0) = x"1" else -- 1: b,c an
73 "110110" when reg_sevensegment(3 downto 0) = x"2" else -- 2: a,b,d,e,g an
74 "111100" when reg_sevensegment(3 downto 0) = x"3" else -- 3: a,b,c,d an
75 "011001" when reg_sevensegment(3 downto 0) = x"4" else -- 4: b,c,f,g an
76 "101101" when reg_sevensegment(3 downto 0) = x"5" else -- 5: a,c,d,f,g an
77 "101111" when reg_sevensegment(3 downto 0) = x"6" else -- 6: a,c,d,e,f,g an
78 "111000" when reg_sevensegment(3 downto 0) = x"7" else -- 7: a,b,c an
79 "111111" when reg_sevensegment(3 downto 0) = x"8" else -- 8: alle an
80 "111101" when reg_sevensegment(3 downto 0) = x"9" else -- 9: a,b,c,d,f,g an
81 "000000"; -- alle aus
82
83 -- Zuweisung der Segmentmuster an die Ausgänge, Berücksichtigung des Dezimalpunkts
84 sevensegment_o <= seg_pattern & (point_status);
85
86
87 end architecture ;

```

Wesentliche Design-Entscheidungen im VHDL-Code:

- **Zeile 48–50:** Der Bus-Handshake (`ack_o`) wird bedingungslos im nächsten Takt erzeugt, sobald `module_active = '1'`. Dies entspricht der im Script (Listing 5, S. 20) gezeigten Methode für schnelle Wishbone-Slaves ohne Wartezyklen.
- **Zeile 52–54:** Der Schreibzugriff übernimmt nur die unteren 8 Bit des 32-Bit-Datenbusses (`dat_i(7 downto 0)`), da das Register 8-Bit-breit ist.
- **Zeile 63–73:** Der kombinatorische Dekoder nutzt `reg_sevensegment(3 downto 0)` für den Vergleich, um die Längen-Diskrepanz zwischen 8-Bit-Register und 4-Bit-Hex-Literalen zu vermeiden.
- **Zeile 76:** Der Ausgang wird zu `seg_pattern & (not point_status)` zusammengesetzt. Aktuell ist `point_status` konstant '0', sodass DP immer ausgeschaltet bleibt (für Common-Kathode: `not '0' = '1' = aus`).

5.3 Integration in das Top-Level

Die Einbindung des Moduls erfordert vier Änderungen in `neorv32_iceduino_top.vhd`, die im Folgenden erläutert werden.

5.3.1 Neues Slave-Response-Signal

Für jedes Wishbone-Slave-Modul muss ein individuelles `slave_resp_t`-Record angelegt werden, damit der Multiplexer die Rückgabedaten korrekt verteilen kann (vgl. Script

Listing 8, S. 21):

```
140
141 -- internal IO connection --
142 -- signal con_gpio : std_ulogic_vector(63 downto 0);
```

5.3.2 Instanziierung des Seven-Segment-Moduls

Das Modul wird mit der generischen Adresse 0xF0000080 instanziiert und an den globalen Wishbone-Bus sowie den Reset angeschlossen:

```
436 generic map (
437     sevensegment_addr      => x"F0000080"
438 )
439 port map (
440     clk_i                  => clk_50mhz,
441     rstn_i                 => external_rstn,
442     adr_i                  => master_bus.adr_o,
443     dat_i                  => master_bus.dat_o,
444     dat_o                  => sevensegment_resp.rdata_i,
445     we_i                   => master_bus.we_o,
446     stb_i                  => master_bus.stb_o,
447     cyc_i                  => master_bus.cyc_o,
448     ack_o                  => sevensegment_resp.ack_i,
449     err_o                  => sevensegment_resp.err_i,
450     sevensegment_o         => io_d,
451     oe_enable              => oe_j6,
452     oe_enable2             => oe_j5
453 );
454
455 -- module instance switch --
```

Die Port-Zuweisung folgt dem im Script (Listing 7, S. 20/21) etablierten Schema: Alle `master_bus.*`-Signale sind für alle Module identisch, die individuellen Response-Signale (`sevensegment_resp`) werden vom Multiplexer verteilt.

5.3.3 Erweiterung des Bus-Multiplexers

Der Multiplexer wurde um den Zweig `when x"80"` erweitert, der die Response-Signale des Seven-Segment-Moduls auf den aktiven Slave-Response-Record schaltet. Die Sensitivity-Liste des Prozesses enthält zusätzlich `sevensegment_resp`:

```
231         active_slave_resp.err_i <= sevensegment_resp.err_i;
232     when others =>
233         active_slave_resp.rdata_i <= (others => '0');
234         active_slave_resp.ack_i <= '0';
```

Wichtig: Die Basisadresse wird an **zwei** Stellen festgelegt: Einmal als generischer Wert bei der Instanz (für die interne `module_active`-Erkennung) und einmal im Multiplexer

(für die Rückführung der Response-Signale). Eine Diskrepanz zwischen diesen beiden Stellen führt zu einem hängenden Buszugriff, da die CPU dann ewig auf `ack_i` wartet.

5.4 Software-Integration

5.4.1 HAL: `iceduino_sevensegment.h` und `iceduino_sevensegment.c`

Die Header-Datei definiert die öffentliche Schnittstelle der Seven-Segment-HAL:

```
1  #ifndef ICEDUINO_SEVENSEGMENT_H
2  #define ICEDUINO_SEVENSEGMENT_H
3
4
5  void iceduino_sevensegment_set(int num);
6  void iceduino_sevensegment_clr();
7  //void iceduino_sevensegment_set_point(int status);
8
9
10 #endif
```

Die Implementation nutzt den in `neorv32_iceduino.h` definierten Memory-Mapped-I/O-Pointer:

```
1  #include "neorv32_iceduino.h"
2
3
4  void iceduino_sevensegment_clr(){
5      ICEDUINO_SEVENSEGMENT = 0x00;
6
7  }
8
9
10 void iceduino_sevensegment_set(int num){
11     ICEDUINO_SEVENSEGMENT = num;
12 }
```

5.4.2 Memory-Mapped-I/O-Adresse

Die Basisadresse und der Zugriffs-Pointer werden zentral in `neorv32_iceduino.h` definiert:

```
45 #define ICEDUINO_PMOD2_OUTPUT (*((volatile uint8_t*) (ICEDUINO_PMOD2_BASE)))
46 //PMOD2 input port 8-bit (r) */
47 #define ICEDUINO_PMOD2_INPUT (*((const volatile uint8_t*) (ICEDUINO_PMOD2_BASE + 8)))
48
49 //PMOD3 address
50 #define ICEDUINO_PMOD3_BASE (0xF0000038UL)
```

Das Schlüsselwort `volatile` ist zwingend erforderlich, um Compiler-Optimierungen bei Hardware-Zugriffen zu verhindern (vgl. Script S. 21). Ohne `volatile` könnte der Compiler aufeinanderfolgende Schreibzugriffe auf dieselbe Adresse als redundant erachten und nur den ersten ausführen.

5.4.3 Anwendungsprogramm: `main.c`

Das Testprogramm zählt zyklisch von 0 bis 9 und schreibt den aktuellen Wert in das Seven-Segment-Register. Durch den Verzicht auf Delays ändert sich der Wert mit maximaler Geschwindigkeit, was in der Simulation gut beobachtbar ist:

```
1 // main.c
2 #include <neorv32_iceduino.h>
3
4 int main(void)
5 {
6     uint8_t i = 0;
7
8     // iceduino_sevensegment_set(0);
9     while (1) {
10         iceduino_sevensegment_set(i); // Schreibe 0..9 ins Register
11         i++;
12         if (i > 9) i = 0;           // 0-9 zählen, ohne Delay!
13
14         for(int i = 0; i<5000000;i++);
15     }
16
17     return 0;
18 }
```

5.5 Build-System-Anpassungen

Damit die neuen Dateien bei Synthese und Simulation berücksichtigt werden, müssen zwei Makefiles angepasst werden.

5.5.1 osflow/boards/iceduino/Makefile

Die VHDL-Quelle des Seven-Segment-Moduls wird der ICEDUINO_SRC-Liste hinzugefügt:

```
62 ICEDUINO_SRC := \  
63 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_led.vhd \  
64 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_switch.vhd \  
65 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_button.vhd \  
66 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_arduino_gpio.vhd \  
67 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_pmod.vhd \  
68 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_arduino_uart.vhd \  
69 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_arduino_spi.vhd \  
70 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_arduino_i2c.vhd \  
71 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_arduino_adc.vhd \  
72 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_sevensegment.vhd \  
73 $(ICEDUINO_HOME)/osflow/boards/iceduino/neorv32_iceduino_top.vhd
```

5.5.2 simulation/Makefile

Analog wird die VHDL-Datei für die GHDL-Simulation ergänzt:

```
49 ICEDUINO_SRC := \  
50 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_led.vhd \  
51 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_switch.vhd \  
52 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_button.vhd \  
53 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_arduino_gpio.vhd \  
54 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_pmod.vhd \  
55 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_arduino_uart.vhd \  
56 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_arduino_spi.vhd \  
57 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_arduino_i2c.vhd \  
58 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_arduino_adc.vhd \  
59 $(ICEDUINO_HOME)/osflow/boards/iceduino/iceduino_sevensegment.vhd \  
60 $(ICEDUINO_HOME)/osflow/boards/iceduino/neorv32_iceduino_top.vhd  
61 DESIGN_NAME := iceduino_tb  
62
```

5.6 Simulation

Die Simulation wurde mit dem Bare-Metal-Testprogramm durchgeführt (INT_BOOTLOADER_EN => false). Nach dem Power-On-Reset (ca. 5 μ s) durchläuft die CPU den Startup-Code crt0.S (ca. 250 μ s) und springt anschließend in die main()-Funktion. Ab ca. 170 μ s sind die ersten Schreibzugriffe auf die Seven-Segment-Adresse 0xF0000080 im Wishbone-Bus sichtbar.

Das Signal reg_sevensegment ändert sich zyklisch mit einem Abstand von ca. 720 ns, was der Ausführungszeit der Schleifeniteration in main.c entspricht (mehrere CPU-Instruktionen pro Schleifendurchlauf bei 50 MHz Takt). Die Zählsequenz 0–9 ist im Trace eindeutig erkennbar, wobei der Wert im Register direkt mit dem unteren Nibble des Dekoders korreliert.

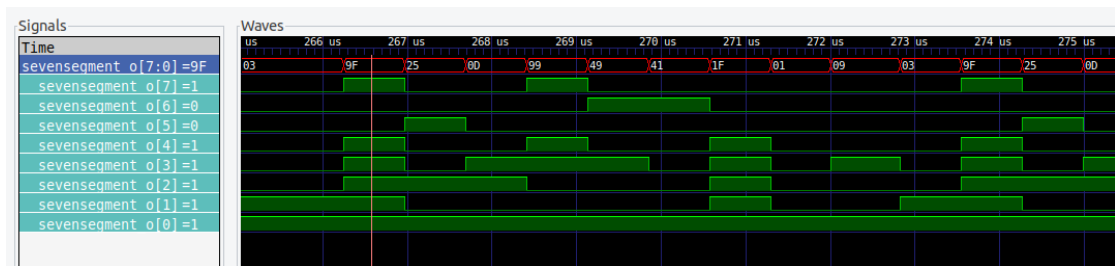


Abbildung 12: GTKWave-Simulation: Zeitlicher Verlauf von `reg_sevensegment` mit Zählsequenz 0–9. Die Änderungen beginnen nach ca. 170 μ s, der Zyklus beträgt ca. 720 ns.

Die Simulation bestätigt die korrekte Funktion der Adressdekodierung und des kombinatorischen Dekoders. Die Ausgänge `sevensegment_o` zeigen die in Tabelle 2 spezifizierten Muster, wobei die Common-Kathode-Logik (aktiv-LOW) berücksichtigt werden muss.

5.7 Pin-Zuweisung und Hardware-Aufbau

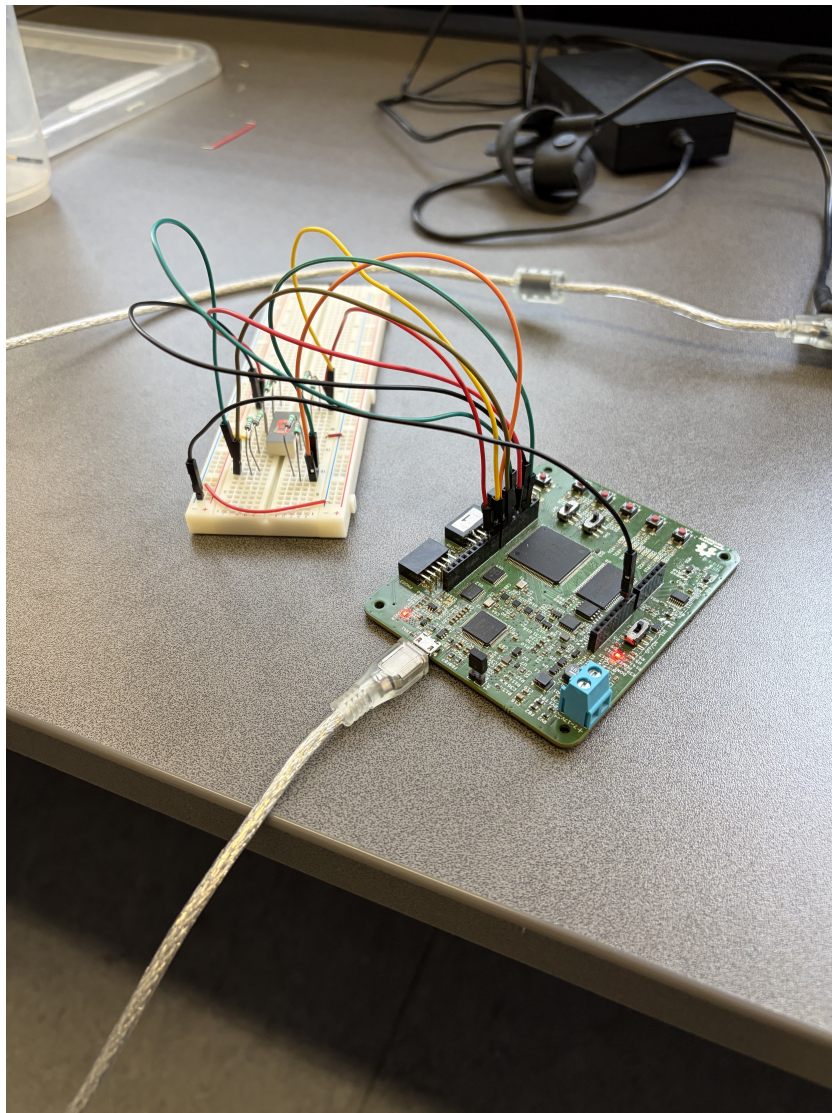


Abbildung 13: Foto des Aufbaus der realen Hardware: FPGA-Board mit angeschlossener 7-Segment-Anzeige. Die Anzeige ist über Vorwiderstände mit den FPGA-Ausgängen verbunden, die Common-Kathode liegt an GND.

Die Pin-Zuweisung erfolgt über die `iceduino.pcf` für freie PMOD- oder Arduino-Header-Pins. Die Sieben-Segment-Anzeige vom Typ Kingbright SC03-12EWA wird mit Vorwiderständen (ca. 220–330 Ohm pro Segment) an die FPGA-Ausgänge angeschlossen, um den Strom zu begrenzen. Die Common-Kathode wird an 0V/GND gelegt, die einzelnen Segmente über die aktiven-HIGH-Ausgänge des FPGA geschaltet. Zur Ansteuerung

wurden die Arduino-Header Pins genutzt. Um die Header zu aktivieren, mussten die zwei *OE5J* und *OE6J* Signale auf HIGH gezogen werden.

6 WS2812 Ansteuerung Entwicklung des Aserial Moduls (Aufgabe 7)

6.1 Implementierung des Aserial Moduls

Um die korrekte Ansteuerung der WS2812 zu gewährleisten, werden die genauen Taktzeiten für die High und Low Phasen berechnet.

Da die 50 MHz Clock genutzt wird, beträgt die Periode 20 ns.

$$T = \frac{1}{50MHz} = 20ns$$

Eine Dauer von 900 ns bzw. 350 ns entspricht demnach folgenden Taktzeiten:

$$\text{Takte für 900 ns} = \frac{900 \text{ ns}}{20 \text{ ns}} = 45 \text{ Takte}$$

$$\text{Takte für 350 ns} = \frac{350 \text{ ns}}{20 \text{ ns}} = 17.5 \text{ Takte}$$

Der Taktzyklus von 17.5 wird auf 18 Perioden aufgerundet.

Ein ganzes Bit dauert theoretisch 1.25 µs, was 62.5 Takten entspricht. Diese Bitzeit wird auf 63 Takte aufgerundet. Grundsätzlich kann eine Ungenauigkeit von 150 ns bzw. 7.5 Taktzyklen toleriert werden.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity ws2812_aserial is
5
6  port(
7      reset : in std_ulogic;
8      clk : in std_ulogic;
9      wnr : in std_ulogic;
10     data_in : in std_ulogic;
11     ws2812_out : out std_ulogic;
12     run : inout std_ulogic
13 );
14
15 end ws2812_aserial;
16
17
18 architecture ws2812_aserial_a of ws2812_aserial is
19
20     signal cnt : integer := 0;
21
22 begin
23
24     process(clk)
25     begin
26         if rising_edge(clk) then
27
```

```

28         if reset = '1' then
29             ws2812_out <= '0';
30             run <= '0';
31             cnt <= 0;
32         elsif wnr = '1' OR run = '1' then
33             if cnt < 18 then
34                 ws2812_out <= '1';
35                 run <= '1';
36                 cnt <= cnt + 1;
37             elsif cnt < 45 then
38                 ws2812_out <= data_in;
39                 run <= '1';
40                 cnt <= cnt + 1;
41             elsif cnt < 63 then
42                 ws2812_out <= '0';
43                 run <= '1';
44                 cnt <= cnt + 1;
45             else
46                 ws2812_out <= '0';
47                 run <= '0';
48                 cnt <= 0;
49             end if;
50         else
51             ws2812_out <= '0';
52             run <= '0';
53             cnt <= 0;
54         end if;
55
56     end if;
57 end process;
58
59 end architecture ws2812_aserial_a;

```

Der VHDL Code implementiert das Aserial Modul, welches die WS2812 LEDs entsprechend der berechneten Taktzeiten ansteuert. Erst nach 350 ns wird das anliegende, zu sendende Bit gelesen um sicherzustellen, dass es aufgrund verzögerter Signale nicht zu Fehlinterpretationen kommt.

6.2 Testbench für das Aserial Modul

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity tb_ws2812 is
6  end entity;
7
8  architecture sim of tb_ws2812 is
9      -- Takt
10     signal clk          : std_logic := '0';
11     constant CLK_PERIOD : time := 20 ns; -- 50 MHz
12
13     -- DUT Signale

```

```

14     signal reset          : std_logic := '1';
15     signal wnr            : std_logic := '0';
16     signal data_in        : std_logic := '0';
17     signal ws2812_out     : std_logic;
18     signal run             : std_logic;
19
20     -- Testbench intern
21     signal data_vector    : std_logic_vector(3 downto 0) := "1010";
22     signal bit_idx        : integer range 0 to 3 := 0;
23
24 begin
25
26     -----
27     -- Taktgenerator (50 MHz = 20 ns Periode)
28     -----
29     clk <= not clk after CLK_PERIOD / 2;
30
31     -----
32     -- DUT Instanziierung
33     -----
34     dut : entity work.ws2812_aserial
35         port map (
36             reset      => reset,
37             clk         => clk,
38             wnr         => wnr,
39             data_in     => data_in,
40             ws2812_out  => ws2812_out,
41             run         => run
42         );
43
44     -----
45     -- Stimulus-Prozess: Sendet "1010" mit korrektem Handshaking
46     -----
47     stim_proc : process
48     begin
49         -- 1. Power-On-Reset (ca. 1 us)
50         reset <= '1';
51         wnr    <= '0';
52         data_in <= '0';
53         wait for 1 us;
54
55         -- 2. Reset freigeben
56         reset <= '0';
57         wait until rising_edge(clk);
58
59         -- 3. Bit-Sequenz "1010" senden
60         for i in 0 to 3 loop
61             -- Warten bis Modul bereit (run = '0')
62
63             wait until rising_edge(clk);
64
65             -- Nächstes Bit anlegen und Senden starten
66             data_in <= data_vector(3-i);
67             wnr      <= '1';
68

```

```

69         -- Warten bis Modul die Übertragung beginnt (run = '1')
70         wait until run = '1';
71
72         wait until run = '0';
73
74         -- WICHTIG: Keine Pause! Nächstes Bit wird sofort
75         -- in der nächsten Iteration angelegt.
76     end loop;
77
78     -- 4. Senden beenden
79     --wait until rising_edge(clk);
80     wnr <= '0';
81
82     -- 5. Zeit für die letzten Flanken lassen (Reset-Code > 50 us)
83     wait for 100 us;
84
85     -- Simulation beenden
86     assert false report "Simulation beendet" severity warning;
87 end process stim_proc;
88
89 end architecture;

```

Die Testbench simuliert die Ansteuerung der WS2812 LEDs indem sie eine Bitfolge von 1010 in MSB-First Reihenfolge sendet. Alle, an das DUT (Device Under Test) angelegten Signale werden in der Testbench definiert und gesteuert, um die Funktionalität des Aserial Moduls zu überprüfen.

6.3 Simulationsergebnisse

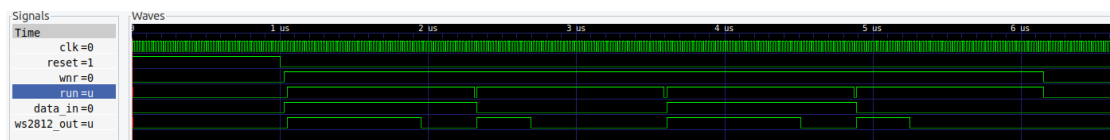


Abbildung 14: Simulation des Aserial Moduls. Die Bitfolge 1010 wird in MSB-First Reihenfolge gesendet.

In der Simulation ist zu erkennen, dass die Bitfolge 1010 korrekt gesendet wird. Im Folgenden werden die einzelnen Bits genauer betrachtet, um die Einhaltung der Timing-Anforderungen zu überprüfen. Hierfür wurden Marker gesetzt, um die Taktflanken zu messen.

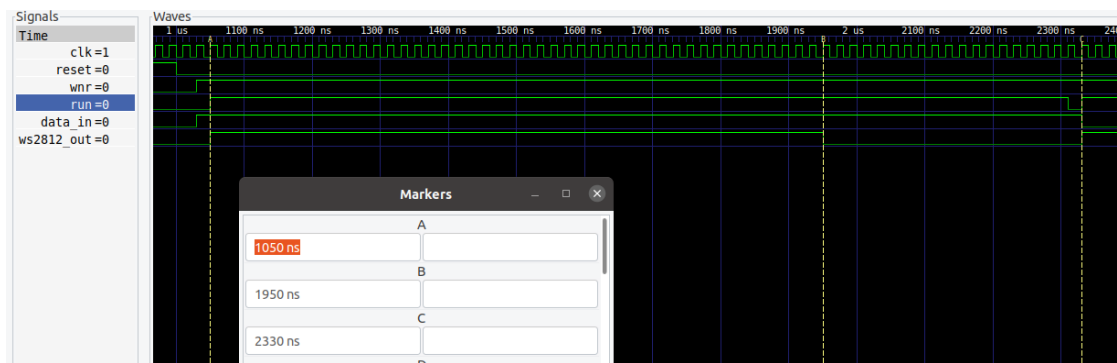


Abbildung 15: Timing des ersten Bits

Aus den Markern ergeben sich Zwischenzeiten von 360 ns und 920 ns, was innerhalb der tolerierbaren Ungenauigkeit liegt.

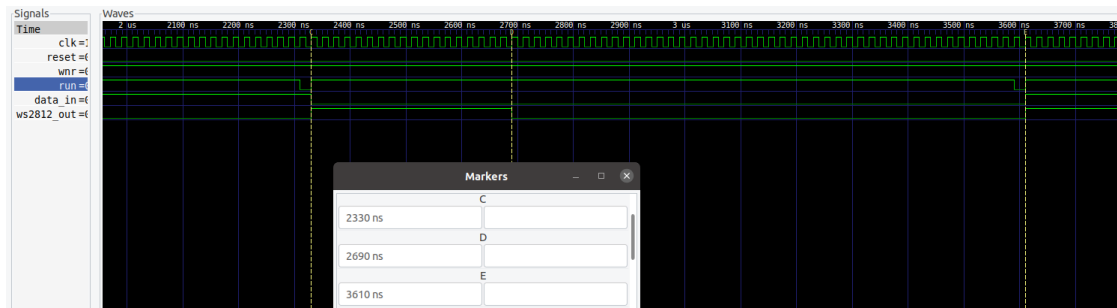


Abbildung 16: Timing des zweiten Bits

Aus den Markern ergeben sich Zwischenzeiten von 900 ns und 380 ns, was innerhalb der tolerierbaren Ungenauigkeit liegt.

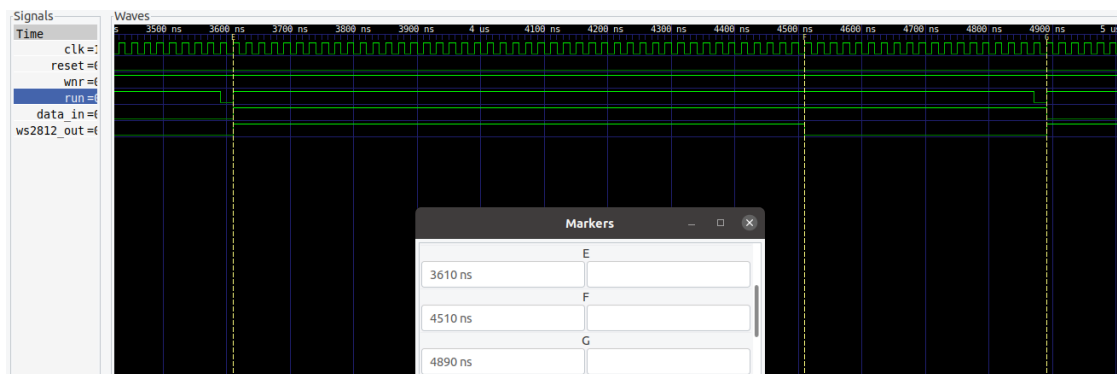


Abbildung 17: Timing des dritten Bits

Aus den Markern ergeben sich Zwischenzeiten von 900 ns und 380 ns, was innerhalb der

tolerierbaren Ungenauigkeit liegt.

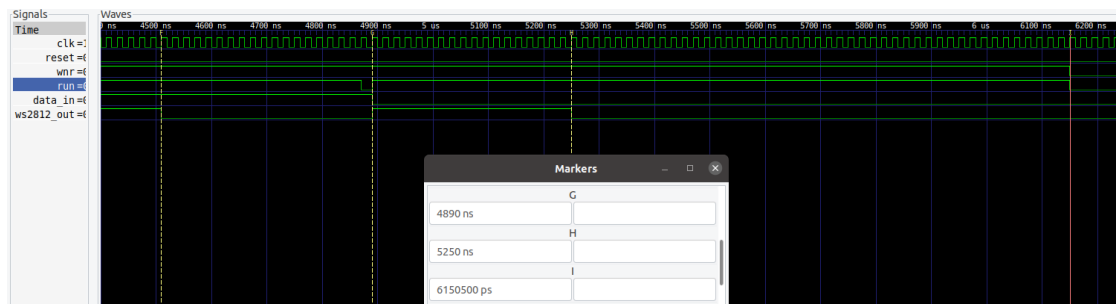


Abbildung 18: Timing des vierten Bits

Aus den Markern ergeben sich Zwischenzeiten von 360 ns und 900.5 ns, was innerhalb der tolerierbaren Ungenauigkeit liegt.